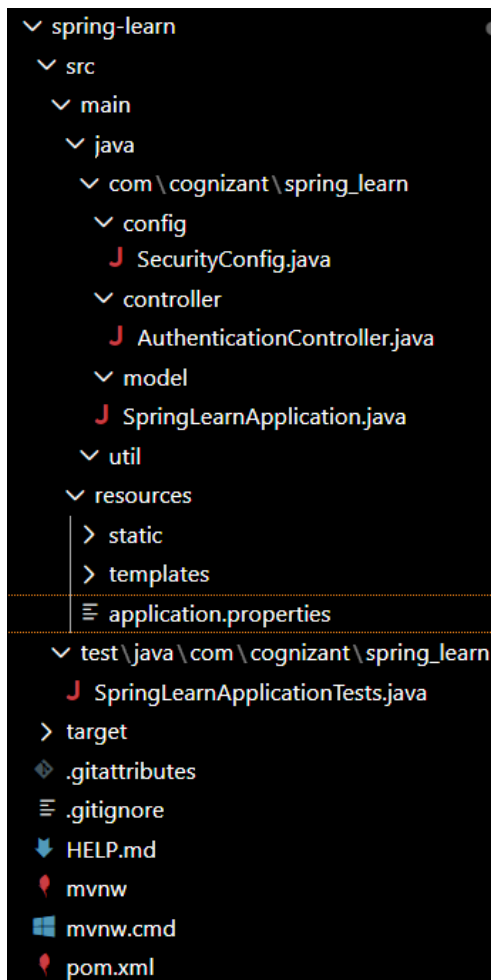


Create authentication service that returns JWT

1. Creation of authentication controller and configuring it in SecurityConfig



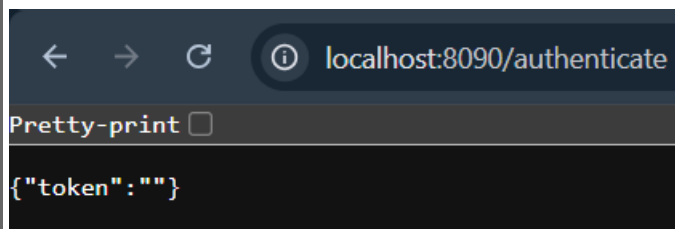
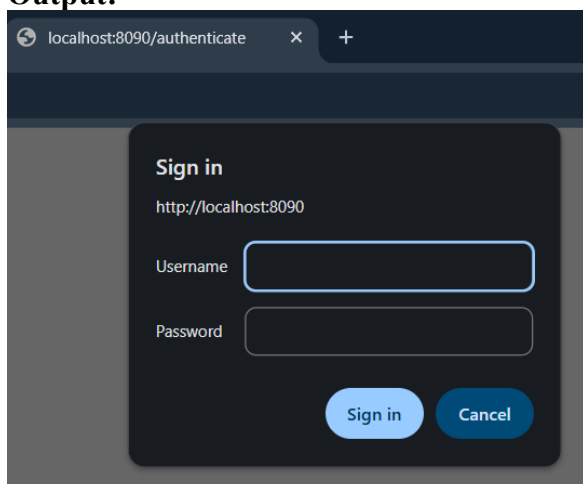
AuthenticationController.java:

```
package com.cognizant.spring_learn.controller;
import java.util.HashMap;
import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class AuthenticationController {
    @GetMapping("/authenticate")
    public Map<String, String> authenticate(
        @RequestHeader("Authorization") String authHeader) {
        System.out.println("START AuthenticationController.authenticate()");
        System.out.println("Authorization Header : " + authHeader);
        Map<String, String> map = new HashMap<>();
        map.put("token", "");
        System.out.println("END AuthenticationController.authenticate()");
        return map;
    }
}
```

SecurityConfig.java:

```
package com.cognizant.spring_learn.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
@Configuration
public class SecurityConfig {
    @Bean
    public InMemoryUserDetailsManager userDetailsService() {
        System.out.println(">>> LOADING CUSTOM USERS <<<");
        UserDetails user = User.builder()
            .username("user")
            .password("{noop}pwd")
            .roles("USER")
            .build();
        UserDetails admin = User.builder()
            .username("admin")
            .password("{noop}pwd")
            .roles("ADMIN")
            .build();
        return new InMemoryUserDetailsManager(user, admin);
    }
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/authenticate")
                .permitAll()
                .anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

Output:



2. Read Authorization header and decode the username and password

AuthenticationController.java:

```
package com.cognizant.spring_learn.controller;
import java.util.Base64;
import java.util.HashMap;
import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class AuthenticationController {
    @GetMapping("/authenticate")
    public Map<String, String> authenticate(
        @RequestHeader("Authorization") String authHeader) {
        System.out.println("START AuthenticationController.authenticate()");
        System.out.println("Authorization Header : " + authHeader);
        String user = getUser(authHeader);
        System.out.println("User : " + user);
        Map<String, String> map = new HashMap<>();
        map.put("token", "");
        System.out.println("END AuthenticationController.authenticate()");
        return map;
    }
    private String getUser(String authHeader) {
        System.out.println("START getUser()");
        String encodedCredentials = authHeader.substring(6);
        System.out.println("Encoded Credentials : " + encodedCredentials);
        byte[] decodedBytes = Base64.getDecoder().decode(encodedCredentials);
        String credentials = new String(decodedBytes);
        System.out.println("Decoded Credentials : " + credentials);
        String user = credentials.split(":")[0];
        System.out.println("Username : " + user);
        System.out.println("END getUser()");
        return user;
    }
}
```

Invocation from authenticate()

```
String user = getUser(authHeader);
System.out.println("User : " + user);
```

Test Performed

Request URL

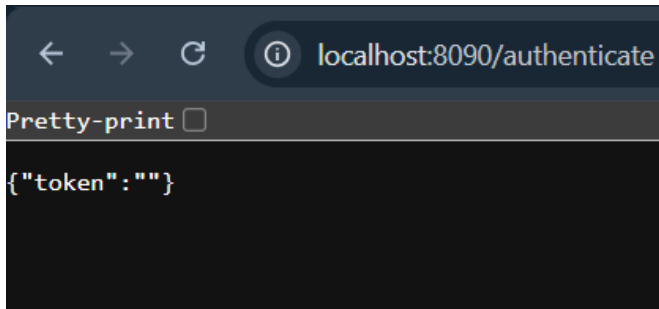
<http://localhost:8090/authenticate>

Authentication Credentials

Username : user

Password : pwd

Output:



```
START AuthenticationController.authenticate()
Authorization Header : Basic dXNlcjpwd2Q=
START getUser()
Encoded Credentials : dXNlcjpwd2Q=
Decoded Credentials : user:pwd
Username : user
END getUser()
User : user
END AuthenticationController.authenticate()
```

3. Generate token based on the user

JWT Generation Method:

```
private String generateJwt(String user) {
    JwtBuilder builder = Jwts.builder();
    builder.setSubject(user);
    // Set the token issue time
    builder.setIssuedAt(new Date());
    // Set the token expiry as 20 minutes from now
    builder.setExpiration(new Date(new Date().getTime() + 1200000));
    builder.signWith(SignatureAlgorithm.HS256, "secretkey");
    String token = builder.compact();
    return token;
}
```

Authentication Method:

```
@GetMapping("/authenticate")
public Map<String, String> authenticate(@RequestHeader("Authorization") String authHeader) {
    LOGGER.info("START AuthenticationController.authenticate()");
    LOGGER.info("Authorization Header : {}", authHeader);
    String user = getUser(authHeader);
    String token = generateJwt(user);
    Map<String, String> map = new HashMap<>();
    map.put("token", token);
    LOGGER.info("END AuthenticationController.authenticate()");
    return map;
}
```

Execution Command

```
$pair="user:pwd"
$encoded=[Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes($pair))
Invoke-RestMethod -Uri "http://localhost:8090/authenticate" -Headers @{Authorization="Basic $encoded"}
```

Output:

```
localhost:8090/authenticate

{"token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIjIjE2VjYiIiwiaWF0IjoxNzg0MDQ2OTM0LCJleHAiOjE3ODQwNDgxMzR9.KEJgNqYhP2WYuwS_qGB0D2ABnxCUI_5p_cy_mf1Gw0c"}
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\spring-learn\spring-learn> $pair="user:pwd"
PS C:\Users\Lenovo\OneDrive\Desktop\spring-learn\spring-learn> $encoded=[Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes($pair))
>>
PS C:\Users\Lenovo\OneDrive\Desktop\spring-learn\spring-learn> Invoke-RestMethod -Uri "http://localhost:8090/authenticate" -Headers @{Authorization="Basic $encoded"}

token
-----
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIjIjE2VjYiIiwiaWF0IjoxNzg0MDQ2OTM0LCJleHAiO...
```

```
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ spring-learn ---
[INFO] Building jar: C:\Users\Lenovo\OneDrive\Desktop\spring-learn\spring-learn\target\spring-learn-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:3.5.16:repackage (repackage) @ spring-learn ---
```